# Assignment-1

★ Aim:

To write a C program to perform basic matrix operations such as addition, subtraction, multiplication & transpose of a matrix using 2-D array

: Objectives:

- To understand the concept of 2-D array in C.
- To perform arithmetic operation on matrices
- To implement matrix addition & subtraction.
- To implement multiplication using nested loops
- To find the Transpose of a matrix.
- To develop programming skills using arrays & loop.

: Theory:

- A matrix is rectangular arrangement of elements organized in rows & cols.
- In C programming, matrices are represented using 2-D arrays.

- Matrix operation are widely used in computer graphics, image processing, ML, scientific computing & engineering applications.

1. Mattix Addition:
   Condition:- Both matrices must have the same number of rows & columns.
   Formula: $C[i][j] = A[i][j] + B[i][j]$

2. Matrix Subtraction:
   Condition: Both matrices must have the same dimensions.
   Formula: $C[i][j] = A[i][j] - B[i][j]$

3. Matrix Multiplication:
   Condition: Number of Cols of 1st matrix must be equal to the No. of rows of 2nd Matrix
   Formula: $C[i][j] = \Sigma A[i][j] \times B[i][j]$

4. Transpose of Matrix:
   It is obtained by interchanging row & cols.
   Formula: $A^T[i][j] = A[j][i]$

# FAQs :-

A1) Matrix addition & subtraction are performed by adding & subtracting the corresponding elements of two matrices.

For ex: $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$

then $A+B = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$

$A-B = \begin{bmatrix} -4 & -4 \\ -4 & -4 \end{bmatrix}$

This can be performed when both matrices have same dimensions.

- Matrix multiplication is performed by multiplying the rows of the first matrix with the colums of 2nd matrix.

- It is possible only when the no. of cols of the first matrix is equal to the no. of rows of 2nd matrix.

- The transpose is obtained by interchanging its rows & cols.

For ex:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

then $A^t = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$

A-2 In C, a matrix is represented by 2-D arrays. A matrix with 3 rows & cols is declared as int A[3][3]. Each element is accessed using its rows & cols index.

- Nested loops are used to input, display & multiply matrix elements. The outer loops controls the rows, while the inner loops controls the cols.

- This allows every element of matrix to be processed efficiently while performing like +, -, * & transpose.

A-3 Matrix multiplication is performed by multiplying the elements of each row of the first matrix with the corresponding element of each column of 2nd matrix.

- The operation is usually implemented by using nested loops, where the 1st loop select the row & 2nd select cols, the third loop selects the sum of product.

- Before multiplication the matrices satisfy compatibility condition. The no. of col of 1st matrix must be equal to the no. of rows in 2nd matrix.

- If this condition satifies, the multiplication can't be performed because corresponding rows, cols elements cant be matched.

A-4) Matrix operations are widely in practical applications. In computer graph they are used to rotate, translate & objects.

- In image processing matrices represent images & help perform operation such as filtering, resizing, etc.

- Matrices are also important in machine learning, storing data & performing calculation in rural network.

- In engineering & scientific computing, they are used to solve systems of eq.s perform stimulation, analyze structure & solve complex mathematical problems.

A-5) Nested loops are essential for matrix operations because a matrix consists of rows & columns.

- The outer loop accesses each row, while the inner loop accesses every coloumn in that row, this allows every element of the matrix to be processed one by one.

- Two nested loops are sufficient for operations such as +, - & transpose matrix multiplication requires three nested loops because an additional loop is required to calculate the sum of product for each element.

# Algorithm:

1. Start
2. Input the no. of rows & column of the first matrix ($r_1, c_1$)
3. Input the no. of rows & column of the second matrix ($r_2, c_2$)
4. Input the elements of 1st matrix A.
5. Input the elements of 2nd matrix B.
6. Display a menu.
7. Input user's choice.
8. Perform selected operations

- Addition :

  If $r_1 = r_2$ & $c_1 == c_2$

  calculate $C[i][j] = A[i][j] + B[i][j]$ using nested loops & display.

- Subtraction:

  If $r_1 == r_2$ & $c_1 == c_2$

  calculate $C[i][j] = A[i][j] - B[i][j]$ using nested loop & print result.

- Multiplication:
  If $c_1 == r_2$,
  Initialize c[i][j] = 0, then
  use 3 nested loops to calculate
  c[i][j] += A[i][k] * B[k][j] &
  display result

- Transpose: Input the matrix to transpose &
  compute T[j][i] = A[i][j] using
  nested loop &
  display result.

9. If the dimensions are incompitable for the selected operations, display error message

10. STOP.

17/7/26

## Code:

```
#include <stdio.h>
void accept(int a[10][10], int m, int n)
{
   int i, j;
   printf("\nEnter matrix elements:\n");
   for (i = 0; i < m; i++)
   {
      for (j = 0; j < n; j++)
      {
         scanf("%d", &a[i][j]);
      }
   }
}

void display(int a[10][10], int m, int n)
{
   int i, j;
   printf("\nMatrix:\n");
   for (i = 0; i < m; i++)
   {
      for (j = 0; j < n; j++)
      {
         printf("%4d", a[i][j]);
      }
      printf("\n");
   }
}

void addition(int a[10][10], int b[10][10], int m, int n)
{
   int c[10][10], i, j;

   for (i = 0; i < m; i++)
   {
      for (j = 0; j < n; j++)
      {
         c[i][j] = a[i][j] + b[i][j];
      }
   }

   printf("\nAddition of matrices:\n");
   display(c, m, n);
}

void subtraction(int a[10][10], int b[10][10], int m, int n)
{
```

```
    int c[10][10], i, j;

    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            c[i][j] = a[i][j] - b[i][j];
        }
    }

    printf("\nSubtraction of matrices:\n");
    display(c, m, n);
}

void multiplication(int a[10][10], int b[10][10], int m, int n)
{
    int c[10][10], i, j, k;

    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            c[i][j] = 0;
            for (k = 0; k < n; k++)
            {
                c[i][j] += a[i][k] * b[k][j];
            }
        }
    }

    printf("\nMultiplication of matrices:\n");
    display(c, m, n);
}

void transpose(int a[10][10], int m, int n)
{
    int t[10][10], i, j;

    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            t[j][i] = a[i][j];
        }
    }

    printf("\nTranspose of matrix:\n");
```

```
    display(t, n, m);
}

int main()
{
    int a[10][10], b[10][10];
    int m, n, ch;

    printf("Enter number of rows and columns: ");
    scanf("%d%d", &m, &n);

    printf("\nEnter First Matrix:\n");
    accept(a, m, n);

    printf("\nEnter Second Matrix:\n");
    accept(b, m, n);

    do
    {
        printf("\n------ MENU ------");
        printf("\n1. Addition");
        printf("\n2. Subtraction");
        printf("\n3. Multiplication");
        printf("\n4. Transpose of First Matrix");
        printf("\n5. Display Matrices");
        printf("\n6. Exit");
        printf("\nEnter your choice: ");
        scanf("%d", &ch);

        switch (ch)
        {
        case 1:
            addition(a, b, m, n);
            break;

        case 2:
            subtraction(a, b, m, n);
            brea

        case 3:
            if (m == n)
                multiplication(a, b, m, n);
            else
                printf("\nMultiplication is supported here only for square matrices.\n");
            break;

        case 4:
```

```
            transpose(a, m, n);
            break;

        case 5:
            printf("\nFirst Matrix:");
            display(a, m, n);

            printf("\nSecond Matrix:");
            display(b, m, n);
            break;

        case 6:
            printf("\nProgram terminated.");
            break;

        default:
            printf("\nInvalid Choice!");
        }

    } while (ch != 6);

    return 0;
}
```

**Input and Output:**

```
Enter number of rows and columns:2 2


Enter First Matrix:

Enter matrix elements:
1 2 3 4


Enter Second Matrix:

Enter matrix elements:
5 6 7 8


------ MENU ------
1. Addition
2. Subtraction
3. Multiplication
4. Transpose of First Matrix
5. Display Matrices
6. Exit
Enter your choice:1
```

```
Addition of matrices:

Matrix:
   6   8
  10  12

------ MENU ------
1. Addition
2. Subtraction
3. Multiplication
4. Transpose of First Matrix
5. Display Matrices
6. Exit
Enter your choice:2


Subtraction of matrices:

Matrix:
  -4  -4
  -4  -4

------ MENU ------
1. Addition
2. Subtraction
3. Multiplication
```

```
3. Multiplication
4. Transpose of First Matrix
5. Display Matrices
6. Exit
Enter your choice:3

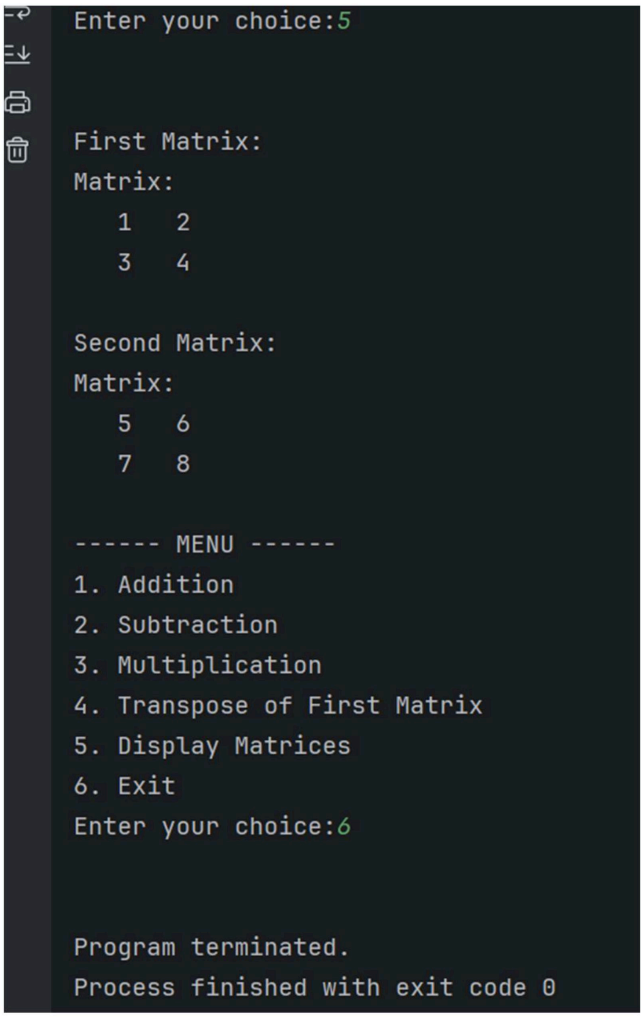

Multiplication of matrices:

Matrix:
  19  22
  43  50

------ MENU ------
1. Addition
2. Subtraction
3. Multiplication
4. Transpose of First Matrix
5. Display Matrices
6. Exit
Enter your choice:4


Transpose of matrix:

Matrix:
   1   3
   2   4
```


Enter your choice:5
First Matrix:
Matrix:
1 2
3 4
Second Matrix:
Matrix:
5 6
7 8
------ MENU ------
1. Addition
2. Subtraction
3. Multiplication
4. Transpose of First Matrix
5. Display Matrices
6. Exit
Enter your choice:6
Program terminated.
Process finished with exit code 0

```
4. Transpose of First Matrix
5. Display Matrices
6. Exit
Enter your choice:5


First Matrix:
Matrix:
   1   2
   3   4

Second Matrix:
Matrix:
   5   6
   7   8

------ MENU ------
1. Addition
2. Subtraction
3. Multiplication
4. Transpose of First Matrix
5. Display Matrices
6. Exit
Enter your choice:6


Program terminated.
Process finished with exit code 0
```